

Обработка на грешки и изключения

опитай...освен...най-накрая

При програмиране в Python може да се срещнат два вида грешки:

Първият тип това са **синтактични грешки**. Те се появяват в резултат на неправилен синтаксис при писане на изходен код. Ако има такива грешки, програмата не може да бъде компилирана. Когато работите в среда за разработка, като PyCharm, самата IDE може да проследява синтактичните грешки и да ги подчертава.

Вторият тип грешки са **грешки по време на изпълнение**. Те се появяват във вече компилирана програма по време на нейното изпълнение. Такива грешки се наричат още изключения. Например преобразуване на низ в число:

```
string = "5"  
number = int(string)  
print(number)
```

Този скрипт ще се компилира и стартира успешно, тъй като низът "5" може лесно да бъде преобразуван в число. Нека обаче вземем друг пример:

```
string = "hello"  
number = int(string)  
print(number)
```

Тема 5. Обработка на грешки и изключения

Този скрипт ще изведе изключение - `ValueError`, тъй като низът „hello“ не може да бъде преобразуван в число:

```
ValueError: invalid literal for int() with base 10: 'hello'
```

Тук е очевидно, че низът не представлява число, но може да се замени с въвеждане от потребителя, който също може да въведе нещо, което не се очаква:

```
string = input("Въведете число: ")  
number = int(string)  
print(number)
```

Когато възникне изключение, програмата се прекъсва и за да се избегне това поведение и да се обработят изключенията, Python има конструкцията `try..except...` `try..except` (опитай освен)

Конструкцията `try..except` има следното формално определение:

```
try:
```

```
    инструкции  
except [тип_на_изключението]:  
    инструкции
```

Тема 5. Обработка на грешки и изключения

Целият код, който потенциално може да предизвика изключение, се поставя след ключовата дума `try`. Ако в този код се изведе изключение, тогава кодът в блока `try` се прекратява и изпълнението отива в блока с изключение. След ключовата дума за изключение може по избор да се посочи кое изключение ще се обработва (например `ValueError` или `KeyError`). След думата с изключение на следващия ред има оператори с изключение на блок, които се изпълняват, когато възникне изключение.

Помислете за обработка на изключения, като използвате примера за преобразуване на низ в число:

```
try:
    number = int(input("Въведете число: "))
    print("Въведено число:", number)
except:
    print("Преобразуването не бе успешно")
print("Край на програмата")
```

Въведете число:

Въведено число: здравейте

Преобразуването не бе успешно

Край на програмата

Тема 5. Обработка на грешки и изключения

Както може да се види от изхода на конзолата, когато се въведе низ, числото не се извежда на конзолата и изпълнението на програмата продължава към блока с изключение. Въведете правилния номер:

Въведете число: 22

Въведено число: 22

Край на програмата

Сега всичко работи нормално, не се хвърля изключение и съответно блокът с изключение не се изпълнява.

накрая блокира

Може също да използвате незадължителен блок `finally`, когато обработвате изключения. Отличителната черта на този блок е, че той се изпълнява независимо от това дали е изведено изключение:

try:

```
number = int(input("Въведете число: "))  
print("Въведено число:", number)
```

except:

```
print("Преобразуването не бе успешно")
```

finally:

```
print("Блокът try приключи изпълнението")  
print("Край на програмата")
```

Тема 5. Обработка на грешки и изключения

Обикновено блок `finally` се използва за освобождаване на използвани ресурси, като например затваряне на файлове, с изключение и обработка на различни видове изключения

Вградени типове изключения

В примера по-горе всички изключения, които могат да възникнат в кода, се обработват наведнъж. Въпреки това може да се посочи типа на изключението, което се обработва, като го посочи след клаузата за изключение:

`try:`

```
number = int(input("Въведете число: "))  
print("Въведено число:", number)
```

except ValueError:

```
    print("Преобразуването не бе успешно")  
print("Край на програмата")
```

Тема 5. Обработка на грешки и изключения

В този случай блокът `except` обработва само изключения от тип `ValueError`, които могат да бъдат изхвърлени, когато низът не е успешно преобразуван в число. Python има следните основни типове изключения:

- **BaseException:** основният тип за всички вградени изключения
- **Изключение:** основният тип, който обикновено се използва за създаване на персонализирани типове изключения
- **ArithmeticError:** основен тип за изключения, свързани с аритметични операции (`OverflowError`, `ZeroDivisionError`, `FloatingPointError`).
- **BufferError:** типът на изключението, което се хвърля, когато операция върху буфера не може да бъде извършена
- **LookupError:** основен тип за изключения, които се издигат при достъп до колекции от невалиден ключ или индекс (например `IndexError`, `KeyError`)

Всички конкретни видове изключения са наследени от тези класове. Python има доста голям списък от вградени изключения. Целият списък може да бъде намерен в документацията. Ето само някои от по-често срещаните:

- **IndexError:** Изключение се хвърля, ако индексът при достъп до елемент от колекцията е извън обхвата.
- **KeyError:** Възниква, ако речникът не съдържа ключ, чрез който се осъществява достъп до елемента на речника.
- **OverflowError:** Вдига се, ако резултатът от аритметична операция не може да бъде представен от текущия числов тип (обикновено `float`).

Тема 5. Обработка на грешки и изключения

- **RecursionError**: Повдига се, ако разрешената дълбочина на рекурсия е надвишена.
- **TypeError**: Вдига се, ако операция или функция се приложи към стойност от невалиден тип.
- **ValueError**: Вдига се, ако операция или функция получи обект от правилния тип с невалидна стойност.
- **ZeroDivisionError**: Повдига се при деление на нула.
- **NotImplementedError**: Тип на изключение, който показва, че някои методи на класа не са внедрени.
- **ModuleNotFoundError**: Повдига се, когато модул не може да бъде намерен, когато е импортиран от директива `import`.
- **OSError**: Типът изключения, които се хвърлят, когато възникнат системни грешки (например файл не може да бъде намерен, дисковата памет е пълна и т.н.)

Тема 5. Обработка на грешки и изключения

И ако ситуацията е такава, че различни типове изключения могат да бъдат хвърлени в програмата, тогава може да ги обработим отделно, като се използват допълнителни оператори с изключение. И когато възникне изключение, Python ще търси правилния блок с изключение, който обработва този тип изключение:

try:

```
number1 = int(input("Въведете първо число: "))  
number2 = int(input("Въведете второ число: "))  
print("Резултат от делението:", number1/number2)
```

except ValueError:

```
print("Преобразуването не бе успешно")
```

except ZeroDivisionError:

```
print("Опит за деление на нула")
```

except BaseException:

```
print("Общо изключение")  
print("Край на програмата")
```

Тема 5. Обработка на грешки и изключения

Ако възникне изключение в резултат на преобразуване на низ в число, то ще бъде обработено от блока `except ValueError`. Ако второто число е равно на нула, т.е. има деление на нула, тогава ще възникне изключение `ZeroDivisionError` и то ще бъде обработено от блока `except ZeroDivisionError`.

Типът `BaseException` представлява общо изключение, под което попадат всички изключения. Следователно в този случай всяко изключение, което не е от тип `ValueError` или `ZeroDivisionError`, ще бъде обработено в `except BaseException`.

Въпреки това, ако програмата хвърли изключение от тип, за който няма съответен блок `except`, програмата няма да може да намери съответния блок с изключение и ще хвърли изключение. Както е посочено в следния случай:

`try:`

```
number1 = int(input("Въведете първо число: "))
```

```
number2 = int(input("Въведете второ число: "))
```

```
print("Резултат от делението:", number1/number2)
```

`except ZeroDivisionError:`

```
print("Опит за деление на нула")
```

```
print("Край на програмата")
```

Тема 5. Обработка на грешки и изключения

Тук разделянето на нула се обработва с помощта на блока `except ZeroDivisionError`. Ако обаче потребителят въведе неконвертируемо число в низ вместо число, тогава ще бъде повдигнато изключение от тип `ValueError`, за което няма съответен блок. И така програмата ще прекъсне изпълнението си.

Python позволява да се обработват множество изключения в един блок с изключение. В този случай всички типове изключения се предават в скоби:

```
try:
    number1 = int(input("Въведете първо число: "))
    number2 = int(input("Въведете второ число: "))
    print("Резултат от делението:", number1/number2)
# обработва два типа изключения - ZeroDivisionError и ValueError
except (ZeroDivisionError, ValueError):
    print("Опит за разделяне на число на нула или неправилно въвеждане")
print("Край на програмата")
```

Получаване на информация за изключение

С оператора `as` може да се предаде цялата информация за изключението към променлива, която след това може да се използва в блока за изключение:

`try:`

```
    number = int(input("Въведете число: "))
```

```
    print("Въведено число:", number)
```

`except ValueError as e:`

```
    print("Сведения за изключения", e)
```

```
print("Край на програмата")
```

Пример за невалиден вход:

Въведете номер: `fdsf`

Подробности за изключение невалиден литерал за `int()` с база 10: `'fdsf'`

Край на програмата

Изхвърляне на изключения и създаване на персонализирани типове изключения

Хвърляне на изключения и изявление за повишаване

Понякога се налага ръчно генериране на изключение. Изявлението за повишаване се използва за това.

Например, нека хвърлим изключение:

```
try:
    number1 = int(input("Въведете първото число: "))
    number2 = int(input("Въведете второто число: "))
    if number2 == 0:
        raise Exception("Второто число не трябва да е 0")
    print("Резултат от деление на две числа:", number1/number2)
except ValueError:
    print("Въведени са некоректни данни ")
except Exception as e:
    print(e)
print("Край на програмата")
```

Тема 5. Обработка на грешки и изключения

Инструкцията за повишаване се предава на обект `BaseException`, в този случай обект `Exception`. Може да предаде съобщение на конструктора от този тип, което след това може да бъде показано на потребителя. В резултат на това, ако `number2=0`, тогава операторът за повишаване ще работи, което ще генерира изключение. В резултат на това програмният контрол ще премине към блока с изключение, който обработва изключения от тип `Exception`:

Въведете първото число: 1

Въведете второто число: 0

Второто число не трябва да е 0

Край на програмата

Създаване на персонализирани типове изключения

В Python няма ограничение до вградени типове изключения и могат да се използват, чрез наследяване, като се създават собствени типове изключения, при необходимост. Ако вземем следния клас `Person`:

Тема 5. Обработка на грешки и изключения

```
class Person:
    def __init__(self, name, age):
        self.__name = name      # установяване на име
        self.__age = age        # установяване на възраст

    def display_info(self):
        print(f'Име: {self.__name} Възраст: {self.__age}')
```

Тук класът `Person` в конструктора получава стойностите за името и възрастта и ги присвоява на личните променливи `име` и `възраст`. Въпреки това, когато се създава обект `Person`, може да се предаде логически неправилна стойност на конструктора – например отрицателно число. Един от начините за решаване на тази ситуация е да се хвърли изключение при предаване на неправилни стойности. Така че нека се дефинира следния програмен код:

Тема 5. Обработка на грешки и изключения

```
class PersonAgeException(Exception):
    def __init__(self, age, minage, maxage):
        self.age = age
        self.minage = minage
        self.maxage = maxage
    def __str__(self):
        return Недопустима стойност: {self.age}. "\
            f"Възрастта трябва да е в диапазона от {self.minage} до {self.maxage}"

class Person:
    def __init__(self, name, age):
        self.__name = name          # установяване на име
        minage, maxage = 1, 110
        if minage < age < maxage:    # установяване на възраст, ако е предадена
            # правилната стойност
            self.__age = age
        else:                        # иначе се генерира изключение
            raise PersonAgeException(age, minage, maxage)
    def display_info(self):
        print(f"Име: {self.__name} Възраст: {self.__age}")
```

Тема 5. Обработка на грешки и изключения

```
try:
    tom = Person("Tom", 37)
    tom.display_info()          # Име: Tom Възраст: 37
    bob = Person("Bob", -23)
    bob.display_info()
except PersonAgeException as e:
    print(e)
```

Недопустима стойност: -23. Възрастта трябва да е в диапазона от 1 до 110

В началото тук е дефиниран класът `PersonAgeException`, който се наследява от класа `Exception`. Обикновено собствените класове изключения се наследяват от класа `Exception`. Класът `PersonAgeException` е за изключения, свързани с възрастта на потребителя. В конструктора `PersonAgeException` получава три стойности - нашата собствена невалидна стойност, която е причинила изключението, както и минималната и максималната стойности за възраст.

```
class PersonAgeException(Exception):
    def __init__(self, age, minage, maxage):
        self.age = age
        self.minage = minage
        self.maxage = maxage

    def __str__(self):
        return f"Недопустима стойност: {self.age}. " \
            f"Възрастта трябва да бъде в диапазон от {self.minage} до {self.maxage}"
```

Тема 5. Обработка на грешки и изключения

Във функцията `__str__` се дефинира текстовото представяне на класа - всъщност съобщението за грешка.

В конструктора на класа `Person` се проверява предадената стойност за възрастта на потребителя. И ако тази стойност не съответства на определен диапазон, тогава се генерира изключение от тип `PersonAgeException`:

```
raise PersonAgeException(age, minage, maxage)
```

Когато се използва класа `Person`, трябва да се вземе предвид, че конструкторът на класа може да хвърли изключение, ако бъде предадена невалидна стойност. Следователно създаването на обекти `Person` е обвито в конструкция `try...except`:

`try:`

```
tom = Person("Tom", 37)
```

```
tom.display_info()          # Име: Tom Възраст: 37
```

```
bob = Person("Bob", -23)    # хвърля се изключение от типа PersonAgeException
```

```
bob.display_info()
```

`except PersonAgeException as e:`

```
print(e) # Недопустима стойност: -23. Възрастта трябва да е в диапазона от 1 до 110
```

И ако при извикване на конструктора `Person` се изведе изключение от типа `PersonAgeException`, тогава контролата на програмата ще отиде в блока с изключение, който обработва изключения от типа `PersonAgeException` под формата на извеждане на информация за изключението към конзолата.

Използвана литература:

- [1]. Joakim Sundnes, Introduction to Scientific Programming with Python, Simula Springer Briefs on Computing, 2020
- [2]. Brian Heinold, A Practical Introduction to Python Programming, Department of Mathematics and Computer Science Mount St. Mary's University, 2012
- [3]. David Mertz, Functional Programming in Python, O'Reilly Media, 2015
- [4]. Steven Lott, Functional Python Programming, Published by Packt Publishing Ltd., 2015
- [5]. Mark Lutz, Learning Python, Fourth Edition, O'Reilly Media, 2009